means that if the user hits Refresh or Reload (perhaps to see whether anyone else has posted a comment), the `add_comment` URL will be sent again to the application. The user intended to refresh the display, but the application sees a request to add another comment. In a blog application, this kind of unintentional double entry is inconvenient. In an online store, it can get expensive.

In these circumstances, the correct way to show the added comment in the index listing is to redirect the browser to the `display` action. We do this using the Rails `redirect_to` method. If the user subsequently hits Refresh, it will simply reinvoke the `display` action and not add another comment.

```
  def add_comment
    @article = Article.find(params[:id])
    comment = Comment.new(params[:comment])
    @article.comments << comment
    if @article.save
      flash[:note] = "Thank you for your valuable comment"
    else
      flash[:note] = "We threw your worthless comment away"
    end
»   redirect_to(action: 'display')
  end
```

Rails has a lightweight yet powerful redirection mechanism. It can redirect to an action in a given controller (passing parameters), to a URL (on or off the current server), or to the previous page.

Let's look at these three forms in turn:

redirect_to(action: ..., options...)

Sends a temporary redirection to the browser based on the values in the `options` hash. The target URL is generated using `url_for`, so this form of `redirect_to` has all the smarts of Rails routing code behind it.

redirect_to(path)

Redirects to the given path. If the path doesn't start with a protocol (such as `http://`), the protocol and port of the current request will be prepended. This method does not perform any rewriting on the URL, so it shouldn't be used to create paths that are intended to link to actions in the application (unless